

RoPE vs ALiBi vs NoPE

Detailed experiment plan & implementation guide — positional encoding ablation on Mini-Qwen (56M params) · Colab/Kaggle T4 free tier

1. Architecture decision: build your own, not HuggingFace

This is the first thing to lock down, because it determines everything downstream. **Use your own Mini-Qwen implementation, not a HuggingFace model.** Here's the reasoning:

- **You need to swap the positional encoding mechanism itself**, not just a config flag. In HF's LlamaModel/Qwen2Model, RoPE is baked into the attention class (`apply_rotary_pos_emb` is called inside `Qwen2Attention.forward`). Ripping that out and inserting an ALiBi bias tensor, or removing positional info entirely for NoPE, means monkey-patching or subclassing internals that were never designed to be swapped. You'd spend more time fighting HF's abstraction than doing science.
- **You already have 90% of this built.** You implemented RoPE from scratch in Mini-Qwen. That means you already have rotary matrix caching, a working attention block, and a training loop. Adding ALiBi is ~20 lines (a bias matrix added to attention logits before softmax). Removing positional info for NoPE is literally deleting a function call. HF gives you none of this shortcut — you'd be reverse-engineering their code to get to the same place.
- **Controlled comparison requires everything else held constant.** The entire point of this experiment is isolating the effect of positional encoding. If you use HF, you inherit their exact tokenizer, weight init scheme, layer norm placement, and MLP activation, which vary across model families and versions — this becomes a confound. With your own model, RoPE, ALiBi, and NoPE variants are identical in every respect except the one line you changed. That's the difference between a publishable ablation and a confused paper reviewers will reject.
- **Reviewers expect this for small-scale ablation papers.** Nearly all ALiBi/NoPE extrapolation papers (Press et al., Kazemnejad et al.) use small custom transformers trained from scratch for exactly this reason — clean, controlled, reproducible. Using a pretrained HF checkpoint and fine-tuning would also contaminate results, since the model has already learned position-dependent behavior from whatever encoding it shipped with.

The only thing worth borrowing from HuggingFace is reference code to sanity-check your ALiBi slope computation against `transformers/models/bloom/modeling_bloom.py` (BLOOM was the original HF model to ship ALiBi) — read it, don't import it.

2. Model architecture — exact spec

Keep the base architecture **frozen and identical** across all 6 runs. Only the positional encoding module changes. Suggested spec (adjust only if your existing Mini-Qwen already fixes these):

Component	Spec
Parameters	~56M (reuse your existing Mini-Qwen config)
Layers	8 transformer blocks
Hidden dim	512

Attention heads	8 (head_dim = 64)
FFN / MLP	SwiGLU, intermediate dim ~ 1376 ($8/3 \times$ hidden, rounded)
Normalization	RMSNorm, pre-norm placement
Attention type	Causal self-attention (no GQA needed at this scale — use MHA for simplicity)
Vocab / tokenizer	GPT-2 BPE tokenizer (50257) — reuse what Mini-Qwen already uses
Positional encoding	SWAPPED per run: RoPE / ALiBi / NoPE (Section 3)

Why hold the architecture this small and fixed: at 56M params on a single T4, a full training run of 50k steps at batch 128 / seq_len 256 takes roughly 1.5-2 hours — this is what makes 6 runs + evals feasible inside free-tier 12-hour session limits, split across 2-3 sessions.

3. Positional encoding — implementation per method

3.1 RoPE (baseline — you have this)

Confirm your existing implementation matches the standard form: rotate query/key pairs by an angle that is a function of absolute position and a per-dimension frequency. Just make sure the rotary cache is precomputed for the max eval length (1024) up front, so extrapolation runs don't silently recompute or clip.

```
class RotaryEmbedding(nn.Module):
    def __init__(self, dim, max_seq_len=1024, base=10000.0):
        super().__init__()
        inv_freq = 1.0 / (base ** (torch.arange(0, dim, 2).float() / dim))
        t = torch.arange(max_seq_len).float()
        freqs = torch.outer(t, inv_freq)  # (max_seq_len, dim/2)
        self.register_buffer("cos", freqs.cos(), persistent=False)
        self.register_buffer("sin", freqs.sin(), persistent=False)

    def forward(self, q, k, seq_len):
        cos, sin = self.cos[:seq_len], self.sin[:seq_len]
        return apply_rotary(q, cos, sin), apply_rotary(k, cos, sin)
```

3.2 ALiBi

No change to Q/K at all. Instead, add a fixed, non-learned bias to the raw attention scores before softmax. The bias penalizes distant key positions linearly, with the penalty slope differing per head (some heads penalize distance harshly, others barely at all).

```

def get_alibi_slopes(n_heads):
    def slopes_power_of_2(n):
        start = 2 ** (-(2 ** -(math.log2(n) - 3)))
        return [start * (start ** i) for i in range(n)]
    return torch.tensor(slopes_power_of_2(n_heads)) # shape (n_heads,)

class ALiBiBias(nn.Module):
    def __init__(self, n_heads, max_seq_len=1024):
        super().__init__()
        slopes = get_alibi_slopes(n_heads).view(n_heads, 1, 1) # (H,1,1)
        pos = torch.arange(max_seq_len)
        rel = pos[None, :] - pos[:, None] # (seq,seq), relative distance
        bias = -rel.abs().float() # penalize distance, negative
        self.register_buffer("bias", (slopes * bias.unsqueeze(0)), persistent=False) # (H,seq,seq)

    def forward(self, attn_scores, seq_len):
        # attn_scores: (batch, n_heads, seq_len, seq_len)
        return attn_scores + self.bias[:, :seq_len, :seq_len].unsqueeze(0)

```

Apply this bias inside the attention forward, right after computing $QK^T / \sqrt{d}$ and before the causal mask + softmax. Q and K get zero positional modification — this is the key structural difference from RoPE.

3.3 NoPE (no positional encoding)

The simplest of the three: delete the positional mechanism entirely. Q and K are unmodified, no bias is added. The only signal the model has about token order is the causal mask itself (position i can only attend to positions $\leq i$). Kazemnejad et al. (2023) showed this implicit signal is enough for a causal decoder to learn some notion of relative order on its own, sometimes generalizing to longer sequences better than RoPE.

```

class NoPositionalEncoding(nn.Module):
    def forward(self, q, k, seq_len):
        return q, k # pass-through, no change

    def bias(self, attn_scores, seq_len):
        return attn_scores # pass-through, no bias added

```

3.4 Wiring it into the config

Add a single flag and dispatch inside your attention block's `__init__` and forward:

```

class Config:
    pos_encoding: str = "rope"    # one of: "rope", "alibi", "nope"

class Attention(nn.Module):
    def __init__(self, config):
        super().__init__()
        if config.pos_encoding == "rope":
            self.rotary = RotaryEmbedding(config.head_dim)
        elif config.pos_encoding == "alibi":
            self.alibi = ALiBiBias(config.n_heads)
        # nope: nothing to instantiate

    def forward(self, x, seq_len):
        q, k, v = self.qkv_proj(x)
        if self.config.pos_encoding == "rope":
            q, k = self.rotary(q, k, seq_len)
        scores = (q @ k.transpose(-2, -1)) / math.sqrt(self.head_dim)
        if self.config.pos_encoding == "alibi":
            scores = self.alibi(scores, seq_len)
        scores = scores.masked_fill(self.causal_mask[:seq_len, :seq_len] == 0, float("-inf"))
        attn = scores.softmax(dim=-1)
        return attn @ v

```

4. Data pipeline

- **Dataset:** WikiText-103 (cleaner, smaller, faster to iterate on Colab than a C4 subset — use C4 only if you want a stronger paper and have compute to spare later).
 - **Tokenizer:** reuse GPT-2 BPE (50257 vocab) — same one your Mini-Qwen already uses, so no retraining a tokenizer.
 - **Packing:** concatenate documents with an EOS separator, then chunk into fixed-length blocks (256 or 512 depending on the run) — standard causal-LM packing, not padding, to avoid wasting compute on pad tokens.
 - **Train/val split:** use WikiText-103's existing train/valid/test split; report all primary metrics on 'valid'.
 - **Extrapolation eval sets:** build two additional held-out chunkings at 768 and 1024 tokens from the same valid split — these are only used at *inference* time on the already-trained models, no retraining involved.
-

5. Training configuration (identical across all 6 runs)

Hyperparameter	Value
Optimizer	AdamW, $\beta_1=0.9$, $\beta_2=0.95$, weight_decay=0.1
LR schedule	Cosine decay with linear warmup (warmup_steps=500)
Peak LR	3e-4
Batch size	128 sequences (use gradient accumulation if T4 VRAM-limited)
Training steps	50,000 (or fewer if val loss plateaus — log and check early)
Precision	fp16 mixed precision (T4 = Turing, no native bf16 support)
Gradient clipping	1.0 (global norm)
Seeds	3 seeds per configuration if compute allows, for confidence intervals; 1 seed minimum viable

Critical: do not tune hyperparameters per-method. Any config difference between RoPE/ALiBi/NoPE runs (other than the pos-encoding module itself) invalidates the comparison. If one method looks unstable, that instability itself may be a valid finding, not a bug to hyperparameter-tune away.

6. Experiment matrix — 6 training runs, 18 evals

Run	Pos. Encoding	Train seq_len	Eval seq_lens
R1	RoPE	256	256 (train), 768, 1024 (extrapolation)
R2	RoPE	512	512 (train), 768, 1024 (extrapolation)
R3	ALiBi	256	256 (train), 768, 1024 (extrapolation)
R4	ALiBi	512	512 (train), 768, 1024 (extrapolation)
R5	NoPE	256	256 (train), 768, 1024 (extrapolation)
R6	NoPE	512	512 (train), 768, 1024 (extrapolation)

6 training runs × 3 eval lengths each = 18 total evaluation numbers for the primary metric table, 12 of which (768 & 1024 columns) constitute the extrapolation study.

7. Metrics — what to log and how

Primary metrics

- **Validation loss / perplexity at train length** — sanity check that all 3 methods actually learn (a broken implementation often shows as a method that never converges).
- **Extrapolation loss / perplexity at 768 and 1024 tokens** — the headline result. Plot as a curve: x-axis = eval sequence length (256/512 train point plus 768, 1024), y-axis = perplexity, one line per method. This is the single most important figure in the paper.

Secondary metrics

- **Attention entropy** — for a sample of eval batches, compute the entropy of each head's attention distribution ($-\sum(p \cdot \log(p))$) over the softmax output, averaged over heads/layers/positions). Low entropy = sharp/focused attention; high entropy = diffuse attention, often a symptom of a model struggling to resolve position at long range.
- **Convergence speed** — steps to reach a fixed target validation loss (pick a threshold from the RoPE baseline run, apply the same threshold to all runs). Tells you if one encoding trains faster, independent of final quality.
- **(Optional, if time allows) Attention heatmaps** — visualize a single head's attention pattern over a fixed input at train length vs. 1024 tokens, side by side across methods. Good qualitative figure for the paper.

8. Analysis & what a result would mean

Outcome	Interpretation
NoPE ties/beats RoPE at train length	Positional encoding may be unnecessary overhead at short context for small models — the causal mask is not needed
ALiBi degrades least at 768/1024	Empirically confirms ALiBi's linear-bias extrapolation advantage holds even at 56M scale, not just in theory
RoPE wins cleanly at both lengths	A clean negative result for ALiBi/NoPE at small scale — still publishable as an empirical validation, and a baseline for future work

9. 3-month timeline

- **Weeks 1-3 — Read & implement.** Read the 5 papers below. Add the `pos_encoding` flag to Mini-Qwen. Implement ALiBi and NoPE. Run short sanity trainings (500-1000 steps) on all 3 to confirm loss goes down for each before committing GPU hours to full runs.
- **Weeks 4-7 — Run all experiments.** 6 full training runs on Colab/Kaggle T4 (~12h total GPU time, split across sessions to respect the 12h session cap). Log everything to Weights & Biases: loss curves, attention entropy snapshots, step-to-target-loss.
- **Weeks 8-10 — Analyze & visualize.** Run all 12 extrapolation evals. Build the main perplexity-vs-length chart. Generate attention heatmaps for the qualitative figure. Draft the results/analysis section.

- **Weeks 11-12 — Write & submit.** Full 6-8 page paper in LaTeX (IEEE or arXiv template). Internal revision pass. Submit to arXiv, then consider IEEE venue given your other paper is already targeting that track.
-

10. Common pitfalls to watch for

- **ALiBi slope bug:** the geometric slope sequence is easy to get subtly wrong. Sanity-check against BLOOM's published slopes for `n_heads=8` before trusting your training curves.
 - **NoPE 'silently learning position from data leakage':** make sure your batches don't always start each sequence at token 0 of a document — if training data has a fixed pattern, NoPE can appear to work only because of leaked positional structure in the data itself, not genuine order-invariant learning.
 - **fp16 NaNs on T4:** if using fp16 (required, since T4 lacks bf16), watch for overflow in the ALiBi bias term at long extrapolation lengths (1024) since the bias magnitude grows with sequence length. Consider computing the bias addition in fp32 even if the rest of the model runs fp16.
 - **Comparing across different effective batch tokens:** keep tokens-per-batch constant across the 256 and 512 `seq_len` runs (e.g. halve the batch size for `seq_len=512`) so all runs see the same amount of data per step.
-

11. Papers to read first

- Su et al., 2021 — *RoFormer: Enhanced Transformer with Rotary Position Embedding* (RoPE)
- Press, Smith & Lewis, 2022 — *Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation* (ALiBi)
- Kazemnejad et al., 2023 — *The Impact of Positional Encoding on Length Generalization in Transformers* (NoPE)
- Touvron et al., 2023 — *Llama 2: Open Foundation and Fine-Tuned Chat Models* (RoPE at production scale)
- Peng et al., 2023 — *YaRN: Efficient Context Window Extension of Large Language Models* (RoPE scaling, useful context for the discussion section)